Convolutional Neural Networks

Week 13 — Learning the filters instead of designing them

Practical Image Processing

Lecture 13 ▪ 60–70 min

The big idea

- Week 3: *we* designed kernels (blur, Sobel) by hand.
- Week 12: hand-designed features (HOG) hit a ceiling.
- Today: let the network **learn the kernels** from data — and learn the classifier at the same time.

Today's real result

A small CNN reads digits (0–9) at **97%** — the DocScan digit reader, trained from scratch.

Roadmap

From kernels to learned filters

Convolution: you already know it

- Week 3: slide a small kernel over the image, multiply-and-sum $\rightarrow$ a filtered image.
- A Sobel kernel detects edges; a blur kernel smooths. *We* chose the numbers.
- A **convolutional layer** is the same operation — but the kernel numbers are **learned**.

Key idea

A CNN is "convolution with kernels chosen by training instead of by hand."

What the network learns



- These are the 8 first-layer 3×3 filters our CNN *learned* for digits.
- Nobody designed them — gradient descent found edge/stroke detectors that help read digits.
- Early layers learn edges; deeper layers combine them into parts, then whole shapes.

Feature maps

- Applying one learned filter to the image gives a **feature map**: where that pattern appears.
- A layer has many filters $\rightarrow$ many feature maps (a "stack" of pattern detectors).
- The next layer convolves over that stack, combining simple patterns into complex ones.

Math

Conv layer: input stack $\rightarrow$ (learned filters) $\rightarrow$ output stack of feature maps.

Anatomy of a CNN

The building blocks

Layer	Does	Why
Convolution	learned filters $\rightarrow$ feature maps	detect patterns
ReLU	$\max(0, x)$, elementwise	add non-linearity
Pooling	downsample (e.g. 2×2 max)	shrink, add invariance
Fully-connected	flatten $\rightarrow$ class scores	make the decision
Softmax	scores $\rightarrow$ probabilities	pick the class

Key idea

Conv+ReLU+Pool blocks stack up, ending in a classifier — the standard CNN recipe.

Why ReLU (non-linearity)?

- Stacking pure convolutions is still just one big linear filter — no extra power.
- **ReLU** ($\max(0, x)$) inserts a non-linearity between layers.
- That lets deep stacks represent complex, curved decision boundaries.

Math

Without a non-linearity, ten conv layers collapse into one. ReLU is what makes "deep" meaningful.

Why pooling?

- **Max-pooling** keeps the strongest response in each small window, halving the size.
- Gives **translation tolerance**: a digit shifted a pixel still activates the same units.
- Shrinks computation and grows the **receptive field** of later layers.

✔ Good practice

Pooling = "I saw this pattern somewhere in this region," not "exactly at this pixel."

Our digit CNN, end to end

- Input 8×8 digit $\rightarrow$ Conv(8 filters)+ReLU+Pool $\rightarrow 4 \times 4$.
- $\rightarrow$ Conv(16 filters)+ReLU+Pool $\rightarrow 2 \times 2 \rightarrow$ flatten (64) $\rightarrow$ FC $\rightarrow$ 10 scores.
- Softmax turns the 10 scores into probabilities; the largest is the predicted digit.

Key idea

Two conv blocks and one linear layer — tiny by modern standards, yet it reads digits at 97%.

Training a CNN

How does it learn the filters?

1. **Forward pass**: run an image through; get predicted probabilities.
2. **Loss**: measure how wrong (**cross-entropy** between prediction and true label).
3. **Backprop**: compute how each weight affected the loss (the gradient).
4. **Update**: nudge every weight to reduce the loss (**gradient descent**).

Key idea

Repeat over many images and epochs; the filters gradually become good detectors.

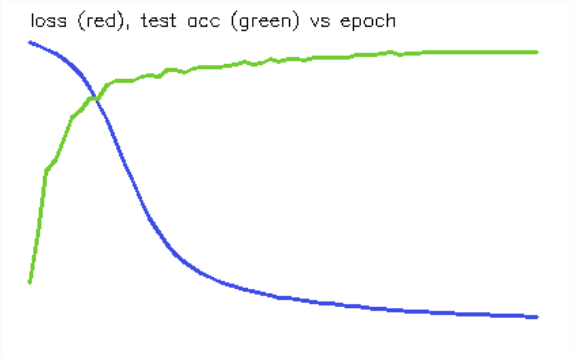
Cross-entropy loss, intuitively

- The network outputs a probability for each class. Cross-entropy is low when it puts high probability on the *correct* class.
- Confidently wrong $\rightarrow$ huge loss. Correct and confident $\rightarrow$ near-zero loss.
- Training minimizes the average loss over the data.

Math

Loss for one sample $= -\log(p_{\text{true class}})$. Push $p_{\text{true}} \rightarrow 1$.

Training in action



- Red: training loss falls each epoch.
- Green: test accuracy climbs, then plateaus near 0.97.
- Watching both curves is how you spot under/over-fitting live.

Mini-batches, epochs, learning rate

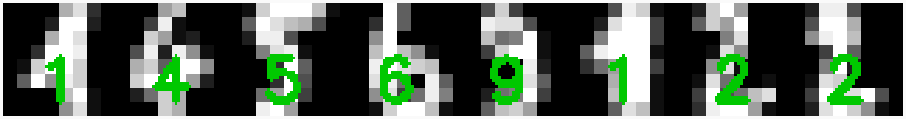
- **Mini-batch**: update on a small group of images at a time (we used 128) — faster, smoother.
- **Epoch**: one full pass over the training set. We ran 60.
- **Learning rate**: step size of each update. Too big → diverges; too small → crawls.

✕ Pitfall

Most training failures are a bad learning rate. If loss explodes or won't move, change it first.

Results

Reading digits — the DocScan payoff



Test digits with the CNN's prediction (green = correct). This is the meter-digit reader DocScan needs.

CNN vs classical — an honest comparison

Method	Test accuracy (digits)
CNN (learned filters)	0.97
SVM on raw pixels	0.99

- On *easy*, clean 8×8 digits, classical ML is already excellent — CNN isn't magic here.
- The CNN's real advantage shows on *hard, large, varied* images — and with **transfer learning** (Week 14).

When do CNNs really win?

- Large, varied datasets where hand features can't capture the variation.
- Problems where the useful features are unknown (the network discovers them).
- Recall Week 12: HOG+SVM plateaued at 48% on look-alike cars — that's where learned features help.

Key idea

Small clean problem $\rightarrow$ classical is fine. Big messy problem $\rightarrow$ deep learning pulls ahead.

In-class exercise

In-class exercise (10 min)

Reason about the architecture

1. How is a convolutional layer related to the Sobel kernel from Week 3?
2. Why won't stacking convolutions *without* ReLU give a more powerful model?
3. Our CNN got 0.97 and the SVM 0.99 on digits. Does that mean CNNs are worse? Explain.

Debrief

- (1) Both convolve a small kernel over the image; Sobel's numbers are fixed, the CNN's are **learned**.
- (2) Composing linear operations is still linear — **ReLU**'s non-linearity is what adds power.
- (3) No — on easy clean data classical ML is already near-perfect; CNNs win on hard, varied, large-scale data and via transfer learning.

Key idea

Match the tool to the problem. The CNN's power is learned features — it pays off when the problem is hard.

Deep dive

The feature hierarchy

- Layer 1: edges and simple strokes (like the learned filters we saw).
- Layer 2–3: corners, curves, digit parts (loops, junctions).
- Deeper: whole-shape detectors that fire for a specific digit.

Key idea

A CNN builds complexity gradually — simple parts combine into meaningful wholes, all learned.

Regularization: keeping CNNs honest

- CNNs have many parameters $\rightarrow$ they can overfit (Week 12's lesson returns).
- **Dropout**: randomly zero some activations during training so the net can't rely on any one.
- **Data augmentation**: random shifts/rotations of training images — free extra data, more invariance.
- **Weight decay**: penalize large weights to keep the model simple.

Why CNNs need a GPU (and lots of data)

- Millions of multiply-adds per image; a GPU does them massively in parallel.
- Deep nets have millions of parameters — they need many examples to fit without overfitting.
- Our digit net is tiny and trains in seconds; ImageNet-scale nets train for days.

✔ Good practice

Small data + deep net = overfitting. Next week's **transfer learning** fixes exactly this.

A CNN in PyTorch

```
class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.c1 = nn.Conv2d(1, 8, 3, padding=1)
        self.c2 = nn.Conv2d(8, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(16*2*2, 10)
    def forward(self, x):
        x = self.pool(torch.relu(self.c1(x)))
        x = self.pool(torch.relu(self.c2(x)))
        return self.fc(x.flatten(1))
```

Conv $\rightarrow$ ReLU $\rightarrow$ Pool, twice, then a linear classifier. That is the whole digit reader.

Wrap-up

What you should now be able to do

- Explain a CNN as convolution with *learned* filters, plus ReLU, pooling, and a classifier.
- Describe how training works: forward, loss, backprop, gradient descent.
- Read training curves and recognize the role of batches, epochs, learning rate.
- Judge honestly when a CNN beats classical ML — and when it doesn't.

This week's deliverable

Due before Week 14 (see `project/`)

1. Train the small digit CNN; report test accuracy and show the training curve.
2. Visualize the learned first-layer filters; describe what they detect.
3. Compare to a classical baseline (SVM/k-NN) on the same digits.
4. Reflect: on your data, when would you expect a CNN to beat classical features?

A CNN learns the filters Week 3 made us design.

Convolution + ReLU + pooling, trained end to end — features and classifier together.